

Chapter 26: Estimation for Software Projects

LECTURE TITLE

Subject: Software
Engineering
Program: BTech Computer
Science and Engineering
Duration: 1 Hour



Software Project Estimation

Moving from Wild Guesses to Educated Estimates

The Universal Stakeholder Questions:

How long will it take?

How much will it cost?

The Engineer's Dilemma:

These are the hardest questions to answer accurately early in a project.



Why We Must Estimate:



Without it, projects cannot be planned.



Budgets cannot be approved.



Resources cannot be allocated.

Lecture Roadmap

Learning Objectives: What You Will Master Today

- **Understand the Landscape:** Explain the core observations and inherent challenges of software estimation.
- **Map the Process:** Describe the software project planning process and evaluate the crucial role of feasibility.
- **Manage the Assets:** Identify and categorize project resources:
 - Human Resources
 - Reusable Components
 - Environmental Resources
- **Break Down the Problem:** Apply systematic decomposition techniques, including:
 - Software sizing
 - Problem-based estimation (LOC and FP)
 - Process-based estimation

The Realities of Software Estimation

Navigating Uncertainty in Software Engineering



The Core Truth:

Estimation is inherently uncertain. The future cannot be perfectly predicted.

Key Observations:



It is an investment: Estimation takes time, but planning is impossible without it.



The Practitioners: Estimates must be done by the people actually doing the work (developers).



Triangulation: Use multiple estimation techniques and compare the results. Investigate any disagreements.



Agility: Initial estimates are rough. They must be updated as the project progresses and knowledge increases.



Learn from the Past: Record historical data to improve future accuracy.

The Project Planning Process

A Step-by-Step Approach to Project Initiation

A systematic approach to moving from an idea to an executable plan:

1. **Establish Scope:** What is explicitly included, and what is excluded?
2. **Assess Feasibility:** Can we actually do this within our current constraints?
3. **Identify Resources:** What people, software, and environments do we need?
4. **Estimate Effort & Cost:** Using decomposition and empirical models.
5. **Develop Schedule:** Map tasks, milestones, and dependencies over time.
6. **Identify Risks:** What could go wrong, and how will we mitigate it?
7. **Plan Quality & Configuration Management:** How will we ensure standards and track changes?

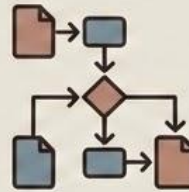
Defining the Software Scope

Bounding the Problem

Before estimating, the scope must define the exact boundaries of the software:



Context: How does this specific software fit into the larger system or business ecosystem?



Information Objectives: What specific data objects are required as inputs and outputs?



Functionality: What exact functions transform the input data into the output data?



Performance: Are there strict constraints on speed, throughput, or response times?

The Go / No-Go Decision

Feasibility Assessment: Evaluating Project Viability

Even if a project is scoped, it must pass four strict feasibility checks before it is approved:



Economic

Does the business case justify the cost of development?



Technical

Can we actually build it with our available technology and team skills?



Operational

Will the end-users actually accept, adopt, and use the system?



Schedule

Can we complete it within the required market timeframes?

Part 2: Resources

Three Categories of Project Resources



Human Resources

- Developers
- Testers
- Technical writers
- Managers
- Support



Reusable Software Resources

- Off-the-shelf
- Reusable components
- Full-experience products



Environmental Resources

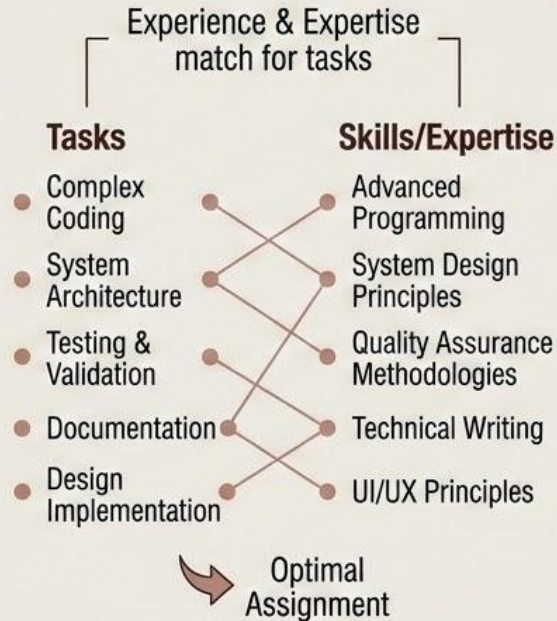
- Hardware
- Software tools
- Network
- Office space
- Cloud services
- Development environment

Human Resources Identification & Skills Matching

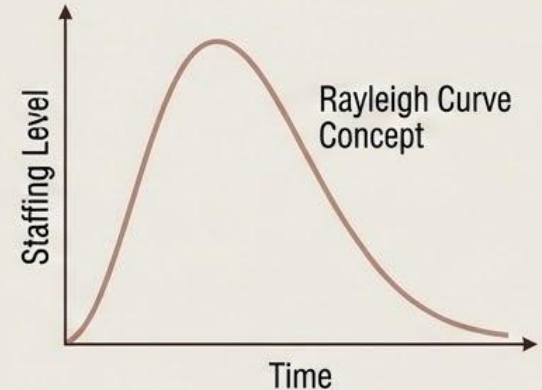
Project Roles



Skills Matching

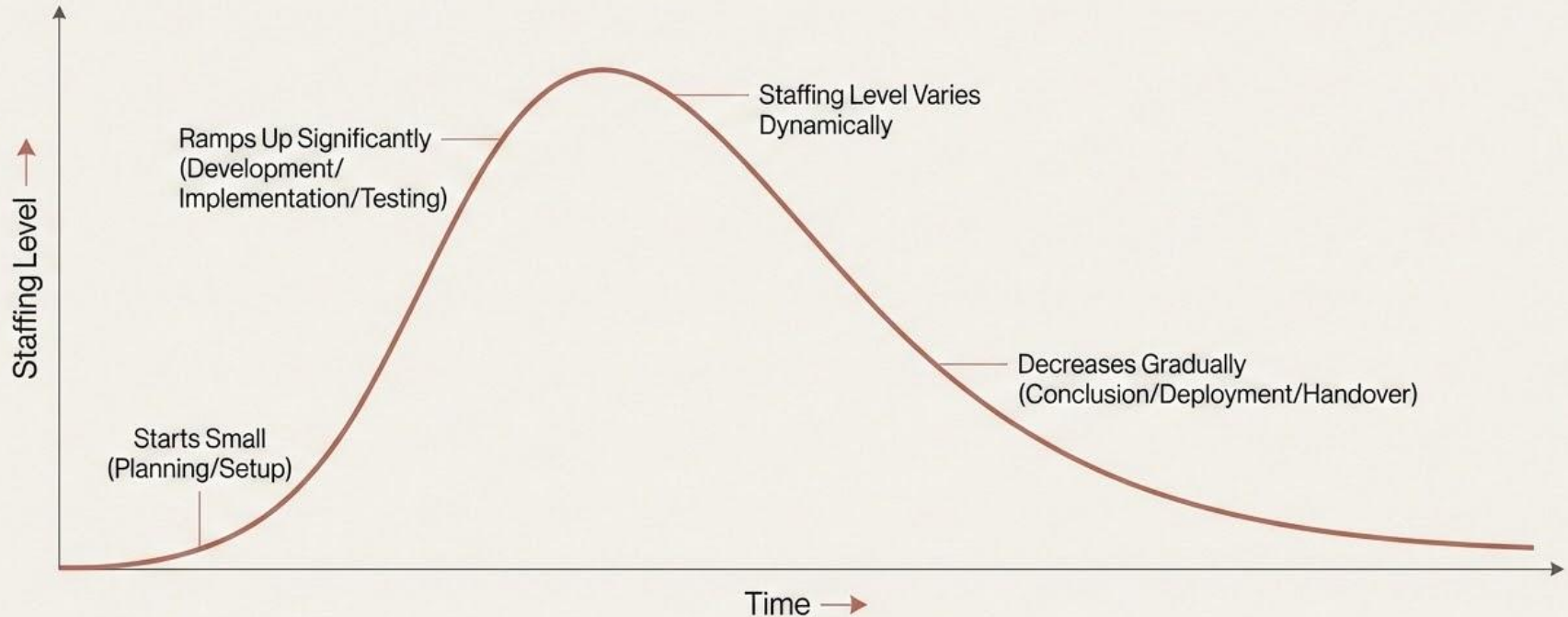


Staffing Dynamics



- Variation over time (Rayleigh curve concept introduction)

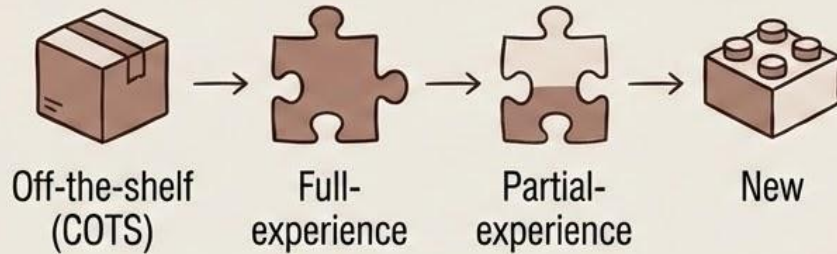
Visualizing Staffing Needs Over Time (The Rayleigh Curve)



Characterized by a characteristic early ramp-up and gradual decline, often following the Rayleigh distribution.

Reusable Components & Environmental Setup

Reusable Component Categories



Evaluation Criteria

- Functional match
- Quality
- Documentation
- Support
- Integration Cost

Environmental Resource Examples



Hardware

- Servers
- Workstations
- Mobile devices



Software

- Tools
- IDEs
- Version control
- Testing/PM tools



Infrastructure

- Network
- Cloud services
- Database servers

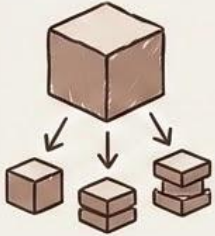


Facilities

- Office space
- Meeting rooms
- Collaboration tools

Software Project Estimation Techniques and the “Cone of Uncertainty”

Two Major Approaches



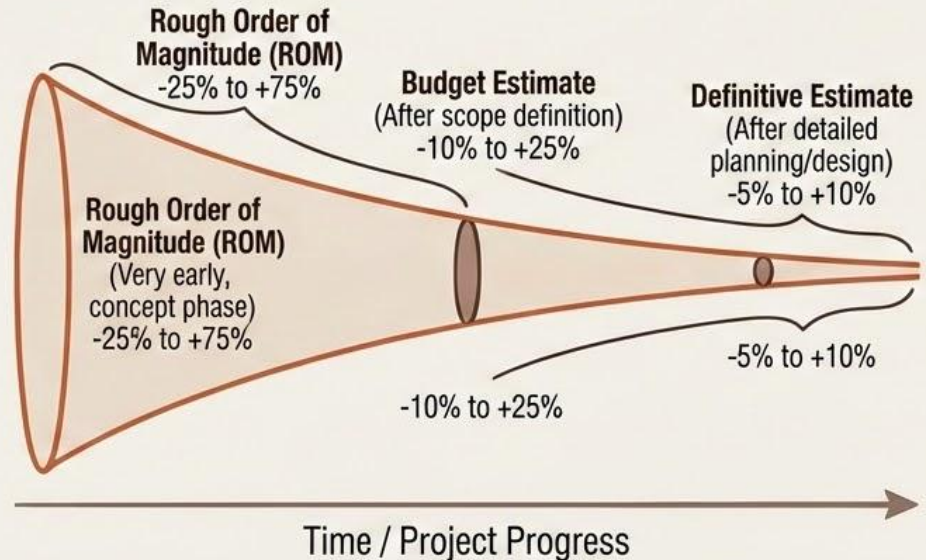
Decomposition Techniques: Break the problem into smaller pieces, estimate each piece individually, and sum them up (Bottom-up).



Empirical Estimation Models: Use mathematical formulas derived from historical project data (e.g., COCOMO).

The Estimation Accuracy Spectrum

Estimates get more accurate as the project progresses and unknowns are resolved.



Software Sizing & Problem-Based Estimation

Quantifying the Work

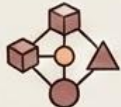
Software Sizing (What are we building?)



Lines of Code (LOC): Direct count of source code lines.



Function Points (FP): Functionality measured from the user's perspective.



Object Points: Used for Object-Oriented or GUI-intensive applications.



Use Case Points: Based on the number and complexity of actors and use cases.

Problem-Based Estimation (How long will it take?)



Step 1: Decompose the problem into features.



Step 2: Estimate the size (LOC or FP) of each feature.



Step 3: Sum to get the total size.



Step 4: Multiply by historical productivity.

The Core Formula:
Effort (person-months) = Estimated Size / Productivity

Estimating with LOC

Lines of Code (LOC) Example



Decomposition & Sizing



User interface (menus, grids, dialogs): 3,000 LOC



Calculation engine (formulas, functions): 4,000 LOC



File I/O (open, save, export): 2,000 LOC



Print module: 1,000 LOC

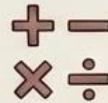
Total Estimated Size:
10,000 LOC



Calculating Effort



Historical Productivity:
500 LOC / person-month



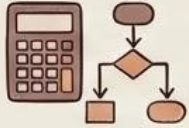
Estimated Effort:
 $10,000 / 500 = 20$ person-months



Duration:
If team size = 4 developers
→ $20 / 4 = 5$ months

Estimating with Function Points

Function Point (FP) Example Focusing on User Value



1. Calculate Unadjusted Function Points (UFP)

Component	Count	Avg Weight	FP Contribution
External Inputs (EI)	15	4	60
External Outputs (EO)	10	5	50
External Inquiries (EQ)	8	4	32
Internal Logical Files (ILF)	3	10	30
External Interface Files (EIF)	2	7	14
Total UFP			186



2. Apply Value Adjustment Factor (VAF)

Assume 14 system complexity factors rated "average" (3 each) → Sum = 42

$$\text{VAF} = 0.65 + (42 / 100) = 1.07$$



3. Calculate Adjusted Effort

Adjusted FP = $186 * 1.07 = 199$ FP
Productivity = 10 FP / person-month



Estimated Effort =

$199 / 10 \approx 20$ person-months

Mapping Effort to Phases

Process-Based Estimation Estimating by Framework Activity



Approach:

Instead of sizing the product, decompose the process into framework activities and estimate the effort required for each phase.

Scenario Example: (Small project: ~100 FP or 5,000 LOC)

Activity	Effort (person-days)	Percentage of Project
 Communication (requirements)	10	8%
 Planning	5	4%
 Modeling (analysis + design)	25	20%
 Construction (coding + unit testing)	60	48%
 Deployment (testing + delivery)	25	20%
Total	125 person-days	100%



Conversion:

$$\boxed{125 \text{ person-days}} \div \boxed{20 \text{ working days/month}} = \boxed{6.25 \text{ person-months}}$$



Triangulation Check & Resolution

Previous Estimate (from LOC/FP examples): **20 person-months** 

This Estimate: 6.25 person-months

Resolution: The previous estimate was for a much larger project (10,000 LOC / ~200 FP). This is a small project. Always reconcile differences.

In-Class Exercise

Decomposing a 'To-Do List' Application

Your Task & Features

-  Add a task
-  Delete a task
-  Mark a task as complete
-  List/View all tasks

Estimation Steps

1. Estimate LOC

How many lines of code do you think each feature will take? Sum them up.



2. Calculate LOC Effort

Assume a productivity of 400 LOC / person-month.



3. Calculate FP Effort (Optional Challenge)

Map features to External Inputs (Add, Delete, Mark) & External Inquiries (List). Assume 12 FP / person-month productivity.



Conclusion & Key Takeaways

Navigating the Essential Uncertainty

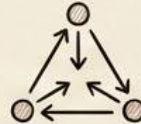


Estimation is Uncertain but Essential: We cannot perfectly predict the future, but we cannot plan or fund a project without trying.

The Golden Rules:



- **Involve the Doers:** Estimates must come from the developers doing the work.

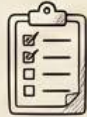


- **Triangulate:** Always use multiple estimation techniques.
- **Iterate:** Update and refine estimates constantly as the project progresses and scope clarifies.

Project Planning & Resources

The Building Blocks of Delivery

The Starting Line



- Rigid Scope Definition
- Four-part Feasibility Assessment (Economic, Technical, Operational, Schedule)

The Three Resource Categories:



Human Resources

People mapped to specific roles and skill levels (accounting for staffing curves).



Reusable Software Resources

COTS products and in-house components (build vs. buy).



Environmental Resources

Hardware, software tools, cloud infrastructure, and physical workspace.

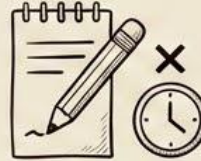
The Mechanics of Measurement

Decomposition Techniques: Breaking Down the Problem



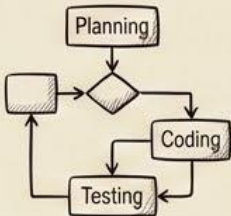
Software Sizing

Measuring the sheer volume of the work (LOC, Function Points, Use Case Points).



Problem-Based Estimation

Estimated Size $\times$ Historical Productivity = Estimated Effort



Process-Based Estimation

Calculating effort based on the individual framework activities (Planning, Coding, Testing, etc.).



Reconciliation

If LOC yields 20 months, FP yields 20 months, and Process-Based 6 months—stop and investigate. Massive differences indicate flawed assumptions or scope misalignment.

The Final Word

The Purpose of Estimation

“All estimates are wrong, but some are useful. The goal is not perfect prediction—it’s to provide enough accuracy for informed decision-making and to improve over time through measurement and learning.”

